ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

| | | |
|---|---|---|
| | | Скобцов Ю.А. |
| должность, уч. степень, звание | подпись, дата | инициалы, фамилия |

# ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 8

Эволюционные алгоритмы оценки стоимости проектов в программной инженерии
Вариант 4
по курсу: Эволюционные методы проектирования программно-информационных систем

РАБОТУ ВЫПОЛНИЛ
СТУДЕНТ

| | | | |
|---|---|---|---|
| гр. № | 4332 | | Могилатов С.И. |
| | | подпись, дата | инициалы, фамилия |

Санкт-Петербург 2026

**СОДЕРЖАНИЕ**

ВВЕДЕНИЕ ..... 3

1 ПОСТАНОВКА ЗАДАЧИ ..... 4

1.1 Исходные данные и вариант ..... 4

1.2 Математическая постановка ..... 4

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ..... 5

2.1 Модель COCOMO ..... 5

2.2 Генетический алгоритм с вещественным кодированием ..... 5

3 АЛГОРИТМ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ..... 7

3.1 Псевдокод ..... 7

3.2 Параметры запуска ..... 7

4 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА ..... 8

КОНТРОЛЬНЫЕ ВОПРОСЫ ..... 10

ЗАКЛЮЧЕНИЕ ..... 12

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ..... 13

ПРИЛОЖЕНИЕ А ..... 14

Программный модуль генетического алгоритма ..... 14

cocomo_ga.py ..... 14

run_lab.py ..... 15

# ВВЕДЕНИЕ

Оценка трудоёмкости проекта на ранней стадии разработки помогает планировать ресурсы и сроки, когда фактические затраты ещё неизвестны. Модель COCOMO связывает размер программного продукта с трудоёмкостью степенной зависимостью E_f=aL^b. Коэффициенты a и b определяют масштаб и скорость роста оценки; их можно настраивать по историческим данным проектов.

Цель лабораторной работы — реализовать генетический алгоритм для поиска коэффициентов модели COCOMO и проверить найденную формулу на отложенных данных. В варианте 4 используются вещественное кодирование, метрика ED, арифметические операторы скрещивания и мутации, а также рулеточный отбор [1]. Отчёт оформлен на титульном бланке ГУАП; параметры текста и структуры сверены с официальными правилами университета [2].

# 1 ПОСТАНОВКА ЗАДАЧИ

## 1.1 Исходные данные и вариант

Использованы 18 проектов из таблицы 8.1 методических указаний [1]. Размер L задан в тысячах строк кода, E_f — в человеко-месяцах. По заданию 13 проектов случайно отбираются для обучения, 5 — для теста; seed=3 фиксирует разбиение.

В проекте №4 напечатано E_f=909,8 человеко-месяца — значение резко выше остальных близких по размеру проектов. Оно сохранено как в источнике и попало в тест; метрики приведены с этой строкой и без неё.

Исходные данные и разбиение на выборки приведены в таблице 1.

Таблица 1 — Данные проектов НАСА

| № | L, тыс. строк | E_f, чел.-мес. |
|---|---|---|
| 1 | 90.2 | 115.8 |
| 2 | 46.2 | 96.0 |
| 3 | 46.5 | 79.0 |
| 4 | 54.5 | 909.8 |
| 5 | 31.1 | 39.6 |
| 6 | 67.5 | 98.4 |
| 7 | 12.8 | 18.9 |
| 8 | 10.5 | 10.3 |
| 9 | 21.5 | 28.5 |
| 10 | 3.1 | 7.0 |
| 11 | 4.2 | 9.0 |
| 12 | 7.8 | 7.3 |
| 13 | 2.1 | 5.0 |
| 14 | 5.0 | 8.4 |
| 15 | 78.6 | 98.7 |
| 16 | 9.7 | 15.6 |
| 17 | 12.5 | 23.9 |
| 18 | 100.8 | 138.3 |

## 1.2 Математическая постановка

Найти положительные a,b модели E_f(L)=aL^b, минимизируя среднеквадратичное евклидово расстояние (ED) на обучающих данных:

$$ED(a,b)=\sqrt{\frac{1}{N}\cdot\sum_{i=1..N}(E_{f,i}-a\cdot L_i^b)^2}.$$

В методических указаниях для параметров не дан численный диапазон. В программе установлены $a\in[0{,}05;\ 20]$ и $b\in[0{,}1;\ 2{,}5]$. Границы обеспечивают положительные коэффициенты и содержат полученные значения; диапазоны являются настройкой эксперимента.

# 2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

## 2.1 Модель COCOMO

COCOMO (Constructive Cost Model) — параметрическая модель оценки затрат на создание программного обеспечения [3]. В базовой форме трудоёмкость выражается как E_f=aL^b, где L — размер кода, а a и b — параметры, определяемые по историческим данным. Поскольку показатели зависят от набора проектов и единиц измерения, их следует калибровать на репрезентативных наблюдениях.

Если взять натуральные логарифмы обеих частей, получаем ln(E_f)=ln(a)+b ln(L). Поэтому для сравнения в работе рассчитывается обычная лог-линейная регрессия. Она минимизирует квадратичную ошибку в логарифмическом пространстве и служит базовым ориентиром, тогда как генетический алгоритм напрямую минимизирует выбранную в варианте метрику ED в исходных единицах.

## 2.2 Генетический алгоритм с вещественным кодированием

Генетические алгоритмы строят популяцию решений и постепенно улучшают её с помощью отбора и вариации кандидатов [4]. В этой работе хромосома — пара вещественных чисел (a,b). Каждое решение оценивается по ED на обучающем множестве. Так как требуется минимизировать ошибку, для рулеточного отбора ошибки преобразуются в положительные веса: вес кандидата равен max(ошибка популяции) − ошибка кандидата плюс малое положительное число. Чем ниже ED, тем больше вероятность стать родителем.

Арифметическое скрещивание двух родителей p и q образует потомка c=αp+(1−α)q, где α равномерно выбирается из [0,1]. При арифметической мутации к каждой координате с вероятностью P_m прибавляется случайное гауссово возмущение с масштабом 6% соответствующего диапазона. Затем значения обрезаются по границам допустимой области.

Элитизм переносит лучшего кандидата без изменения. Фиксированное число поколений даёт сопоставимую историю сходимости между запусками.

# 3 АЛГОРИТМ И ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

## 3.1 Псевдокод

1. Считать 18 проектов и перемешать индексы фиксированным seed.
2. Выделить 13 записей для обучения и 5 для теста.
3. Случайно создать популяцию пар (a,b) в установленных границах.
4. Для каждого кандидата вычислить E_f=aL^b и значение ED на обучающих строках.
5. Выбрать родителей рулеткой по весам, обратно связанным с ошибкой.
6. С вероятностью P_c выполнить арифметическое скрещивание; с вероятностью P_m мутировать каждую координату.
7. Ограничить параметры, сохранить элиту и повторить шаги 4–7 для всех поколений.
8. Выбрать наилучшую формулу по обучающей ED, оценить её на пяти отложенных проектах, сохранить таблицы и графики.

## 3.2 Параметры запуска

Выполнено 8 независимых запусков. В каждом использована популяция 120 решений, 600 поколений, P_c=0,90 и P_m=0,15. Начальное состояние генератора менялось между запусками. Для итоговой модели выбрана особь с минимальной обучающей ED среди восьми запусков; тестовые значения при выборе не использовались.

Таблица 2 фиксирует настройки и правила воспроизводимости эксперимента.

Таблица 2 — Параметры генетического алгоритма

| Параметр | Значение |
|---|---|
| Кодирование | Вещественная пара (a,b) |
| Размер популяции | 120 |
| Число поколений | 600 |
| Вероятность скрещивания P_c | 0,90 |
| Вероятность мутации P_m | 0,15 на ген |
| Отбор | Рулеточный по преобразованным весам |
| Элитизм | 1 лучший кандидат |
| Границы a | [0,05; 20] |
| Границы b | [0,1; 2,5] |
| Разбиение обуч./тест | 13 / 5, seed=3 |
| Независимые запуски | 8; seeds 42–49 |

## 4 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА

Лучший из восьми запусков дал a=2.811018, b=0.838826; итоговая формула: Ef = 2.811018 * L^0.838826. Обучающая ED составила 9.657 человеко-месяца. Во всех запусках найденные параметры близки друг к другу, что указывает на устойчивость данного поиска для выбранного обучающего набора.

На рисунке 1 показано снижение лучшей обучающей ошибки по поколениям. Таблица 3 содержит значения параметров и ED для каждого независимого запуска.

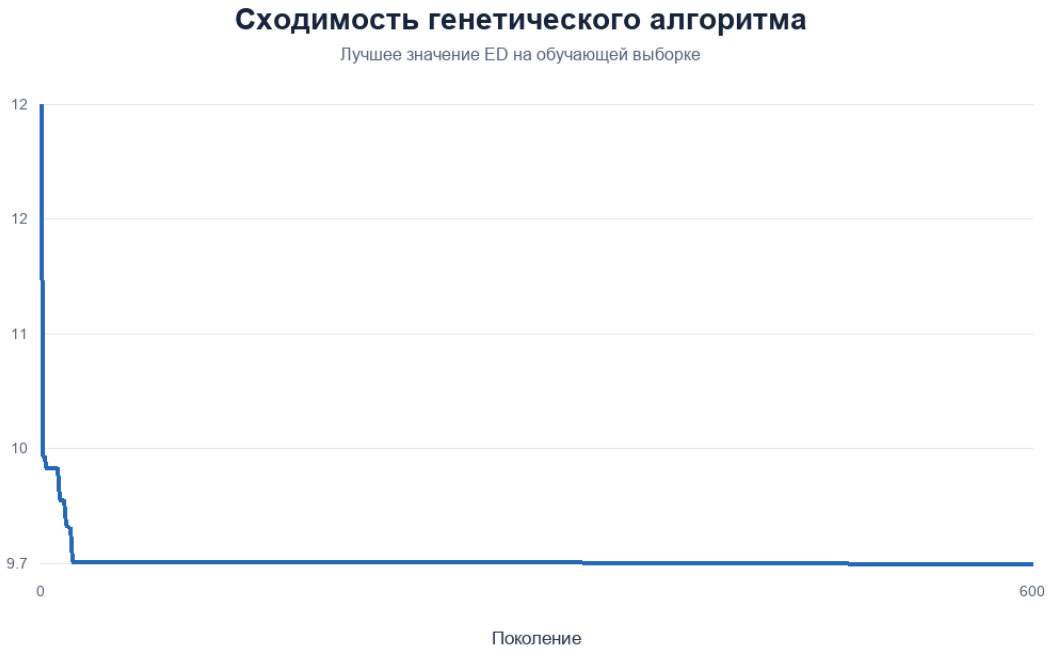



Рисунок 1 — Сходимость лучшей обучающей ошибки ED

Таблица 3 — Восемь независимых запусков

| Seed | a | b | ED обучения |
|---|---|---|---|
| 42 | 2.803669 | 0.839448 | 9.6568 |
| 43 | 2.912132 | 0.829877 | 9.6636 |
| 44 | 2.827571 | 0.837423 | 9.6568 |
| 45 | 2.841734 | 0.836338 | 9.6570 |
| 46 | 2.806560 | 0.839237 | 9.6568 |
| 47 | 2.811018 | 0.838826 | 9.6567 |
| 48 | 2.833327 | 0.836999 | 9.6569 |
| 49 | 2.819559 | 0.838214 | 9.6568 |

Тестовая выборка содержит проекты №4, 6, 7, 9 и 16. Таблица 4 сопоставляет фактическую трудоёмкость и прогноз итоговой модели.

Таблица 4 — Прогноз на тестовом множестве

| Проект | L | Факт, чел.-мес. | Прогноз GA | \|Ошибка\| |
|---|---|---|---|---|
| 4 | 54.5 | 909.8 | 80.43 | 829.37 |
| 6 | 67.5 | 98.4 | 96.23 | 2.17 |
| 7 | 12.8 | 18.9 | 23.86 | 4.96 |
| 9 | 21.5 | 28.5 | 36.86 | 8.36 |
| 16 | 9.7 | 15.6 | 18.91 | 3.31 |

Рисунок 2 сравнивает фактические и прогнозные значения на обучающих и тестовых проектах для генетической модели и лог-линейной базовой модели. Логарифмические оси позволяют увидеть обычные наблюдения рядом с необычно крупным значением проекта №4.

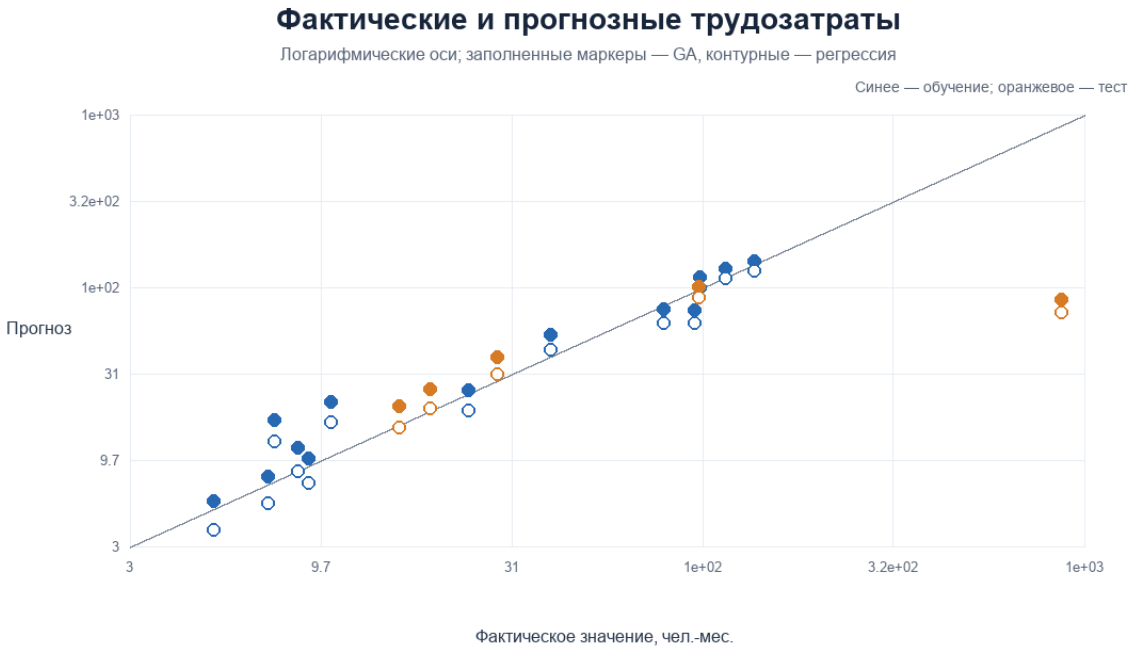



Рисунок 2 — Фактические и прогнозные трудозатраты

Таблица 5 сравнивает среднеквадратичную абсолютную ошибку ED и среднюю относительную ошибку MMRE. Для генетической модели отдельно показан тест без аномального проекта №4.

Таблица 5 — Сравнение качества модели

| Модель и множество | ED, чел.-мес. | MMRE, % |
|---|---|---|
| GA, обучение | 9.657 | 26.14 |
| GA, тест (все 5) | 370.937 | 34.02 |
| GA, тест без проекта 4 | 5.246 | 19.74 |
| Лог-линейная регрессия, обучение | 10.150 | 23.16 |
| Лог-линейная регрессия, тест | 372.848 | 24.76 |

Тестовая ED велика из-за проекта №4 (факт 909,8; прогноз около 80,4). Без него ED остальных четырёх тестовых проектов равна 5,246 человеко-месяца.

## КОНТРОЛЬНЫЕ ВОПРОСЫ

**1. Приведите основную формулу методики COCOMO.**

Ответ: E_f=aL^b, где L — размер программного продукта в тысячах строк кода, а E_f — оценка трудоёмкости в человеко-месяцах. Коэффициенты a и b калибруются по историческим проектам.

**2. Какие данные необходимы для использования COCOMO?**

Ответ: Нужны размер проекта L и фактическая трудоёмкость E_f для проектов сравнимого класса. Для калибровки параметры подбираются по парам этих измерений.

**3. Что позволяет оценивать методика COCOMO?**

Ответ: Она оценивает трудоёмкость разработки программного обеспечения по размеру кода. Полученную оценку можно применять при планировании затрат и ресурсов.

**4. Какие данные можно использовать при обучении (поиске коэффициентов a,b)?**

Ответ: Набор исторических проектов с измеренными L и E_f. В этой работе использованы данные 18 проектов из таблицы 8.1.

**5. Как можно разбить данные на обучающее и тестовое множество?**

Ответ: Случайно разделить наблюдения на непересекающиеся подмножества, используя обучающую часть для настройки параметров, а тестовую — только для независимой оценки. В работе использовано 13 и 5 проектов соответственно.

**6. Какое кодирование потенциальных решений можно использовать в этой задаче?**

Ответ: Для генетического алгоритма — вещественную хромосому (a,b), как в варианте 4. Также возможно двоичное кодирование после дискретизации диапазонов параметров, но оно здесь не применяется.

**7. Какие типы генетических операторов можно использовать в этой задаче?**

Ответ: Отбор родителей, скрещивание и мутацию. В варианте 4 заданы рулеточный отбор, арифметическое скрещивание и арифметическая вещественная мутация.

**8. Какую фитнес-функцию можно использовать в этой задаче?**

Ответ: Для минимизации расхождения между измеренной и расчётной трудоёмкостью используется ED — корень из среднего квадрата абсолютных остатков в человеко-месяцах.

**9. Какие виды ошибок можно использовать в этой задаче?**

Ответ: Абсолютную (сумма модулей остатков), относительную (например, MRE/MMRE), среднеквадратичную ошибку и метрики разброса/объяснённой дисперсии. Выбор зависит от того, важнее ли абсолютная или относительная точность.

**10. Приведите формулу метрики абсолютных значений MD.**

Ответ: $MD=\Sigma_i|E_f,i-E_fm,i|$, где $E_f,i$ — фактическая трудоёмкость, а $E_fm,i$ — рассчитанная моделью оценка i-го проекта.

**11. Приведите формулу средней относительной погрешности.**

Ответ: $MMRE=(1/N)\Sigma_i\ |E_f,i-E_fm,i|/|E_f,i|$. В процентах результат умножается на 100. Наблюдения с нулевой фактической трудоёмкостью требуют отдельной обработки.

**12. Приведите формулу отклонения (дисперсии) из методички.**

Ответ: Согласно приведённым указаниям, $VAF=[1-Var(E_f-E_fm)/Var(E_f)]\cdot 100\%$. Здесь Var — дисперсия соответствующих значений. В отчёте используется именно обозначение и формула из задания.

**13. Как можно улучшить методику COCOMO?**

Ответ: Калибровать параметры на актуальном и репрезентативном наборе, учитывать тип проекта и факторы сложности, проверять выбросы и качество измерений, разделять данные на обучение и тест, а качество оценивать несколькими метриками.

**14. Как применить роевой алгоритм для поиска a,b?**

Ответ: Положение частицы задать парой (a,b), а целевой функцией сделать ED на обучающих данных. Частицы перемещать по стандартным формулам PSO, ограничить параметры положительной областью и выбрать позицию с минимальной ошибкой.

**15. Как применить эволюционную стратегию для поиска a,b?**

Ответ: Начать с популяции вещественных векторов (a,b), создавать потомков рекомбинацией и мутацией, оценивать их по ED и формировать следующую популяцию отбором лучших. Повторять до достижения критерия остановки.

Полный листинг генетического алгоритма и программы запуска приведён в приложении А.

## ЗАКЛЮЧЕНИЕ

Выполнена калибровка модели COCOMO генетическим алгоритмом с вещественным кодированием и операторами варианта 4. Для итоговой формулы E_f=2.8110·L^0.8388 обучающая ED составила 9.657 человеко-месяца.

На фиксированной тестовой выборке ошибка ED равна 370,937 человеко-месяца с проектом №4 и 5,246 без него. Большая разница вызвана исходным значением 909,8, перенесённым в отчёт как напечатано. Следовательно, вывод о качестве калибровки зависит от проверки качества исходных данных; аномальную строку следует уточнить у составителя таблицы, не подменяя её предположительным значением.

Восемь повторов дали близкие коэффициенты и обучающие ошибки, а лог-линейный ориентир показал сопоставимый результат. Эксперимент демонстрирует корректную работу генетической настройки, но малая выборка и выброс ограничивают обобщение результатов.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторная работа № 8 «Эволюционные алгоритмы оценки стоимости проектов в программной инженерии»: методические указания. — Материал, предоставленный для выполнения работы.

2. ГУАП. Для учебного процесса: нормативные требования и бланки титульных листов. — URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 10.11.2026).

3. Boehm B. W. Software Engineering Economics. — Englewood Cliffs: Prentice-Hall, 1981.

4. Haupt R. L., Haupt S. E. Practical Genetic Algorithms. 2nd ed. — Hoboken: John Wiley & Sons, 2004.

# ПРИЛОЖЕНИЕ А

## Программный модуль генетического алгоритма

### cocomo_ga.py

```
"""Real-valued genetic algorithm for the two-parameter COCOMO model."""
from __future__ import annotations


from dataclasses import dataclass
import math
import random
from typing import Callable, Sequence


A_BOUNDS = (0.05, 20.0)
B_BOUNDS = (0.1, 2.5)



@dataclass(frozen=True)
class FitResult:
    a: float
    b: float
    error: float
    history: tuple[float, ...]



def cocomo_effort(kloc: float, a: float, b: float) -> float:
    if kloc <= 0 or a <= 0:
        raise ValueError("KLOC and coefficient a must be positive")
    return a * kloc**b



def absolute_error_rms(actual: Sequence[float], predicted: Sequence[float]) -> float:
    if len(actual) != len(predicted) or not actual:
        raise ValueError("actual and predicted must have the same non-zero length")
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(actual, predicted)) / len(actual))



def predict(kloc: Sequence[float], a: float, b: float) -> list[float]:
    return [cocomo_effort(size, a, b) for size in kloc]



def _score(candidate: tuple[float, float], kloc: Sequence[float], effort: Sequence[float]) -> float:
    try:
        predictions = predict(kloc, *candidate)
        if not all(math.isfinite(value) for value in predictions):
            return math.inf
        return absolute_error_rms(effort, predictions)
    except (OverflowError, ValueError):
        return math.inf



def _roulette(population: list[tuple[float, float]], errors: list[float], rng: random.Random) -> tuple[float, float]:
    finite = [e for e in errors if math.isfinite(e)]
    if not finite:
        return rng.choice(population)
    worst = max(finite)
    weights = [max(0.0, worst - e) + 1e-12 if math.isfinite(e) else 0.0 for e in errors]
    return rng.choices(population, weights=weights, k=1)[0]



def fit_genetic_algorithm(
    kloc: Sequence[float],
    effort: Sequence[float],
    *,
    population_size: int = 100,
    generations: int = 400,
    crossover_probability: float = 0.9,
```

```
    mutation_probability: float = 0.15,
    seed: int = 42,
    a_bounds: tuple[float, float] = A_BOUNDS,
    b_bounds: tuple[float, float] = B_BOUNDS,
    on_generation: Callable[[int, float], None] | None = None,
) -> FitResult:
    """Minimize the RMS/ED on a training set using roulette selection and real coding."""
    if len(kloc) != len(effort) or len(kloc) < 2:
        raise ValueError("at least two paired observations are required")
    if any(x <= 0 for x in kloc) or any(y < 0 for y in effort):
        raise ValueError("KLOC must be positive and effort non-negative")
    if population_size < 4 or generations < 1:
        raise ValueError("population_size must be >= 4 and generations >= 1")
    if not 0 <= crossover_probability <= 1 or not 0 <= mutation_probability <= 1:
        raise ValueError("operator probabilities must be in [0, 1]")
    if not (0 < a_bounds[0] < a_bounds[1] and 0 < b_bounds[0] < b_bounds[1]):
        raise ValueError("parameter bounds must be positive and increasing")


    rng = random.Random(seed)
    bounds = (a_bounds, b_bounds)
    spans = tuple(high - low for low, high in bounds)
    population = [tuple(rng.uniform(low, high) for low, high in bounds) for _ in range(population_size)]
    history: list[float] = []


    for generation in range(generations + 1):
        errors = [_score(individual, kloc, effort) for individual in population]
        best_index = min(range(population_size), key=errors.__getitem__)
        best_error = errors[best_index]
        history.append(best_error)
        if on_generation:
            on_generation(generation, best_error)
        if generation == generations:
            break


        next_population = [population[best_index]]  # elitism protects the current best
        while len(next_population) < population_size:
            parent1 = _roulette(population, errors, rng)
            parent2 = _roulette(population, errors, rng)
            if rng.random() < crossover_probability:
                alpha = rng.random()
                child = tuple(alpha * p1 + (1.0 - alpha) * p2 for p1, p2 in zip(parent1, parent2))
            else:
                child = parent1
            genes = list(child)
            for gene_index, (low, high) in enumerate(bounds):
                if rng.random() < mutation_probability:
                    # Arithmetic mutation: add a bounded real-valued perturbation.
                    genes[gene_index] += rng.gauss(0.0, spans[gene_index] * 0.06)
                genes[gene_index] = min(high, max(low, genes[gene_index]))
            next_population.append((genes[0], genes[1]))
        population = next_population


    best_index = min(range(population_size), key=lambda i: _score(population[i], kloc, effort))
    best_a, best_b = population[best_index]
    return FitResult(best_a, best_b, _score((best_a, best_b), kloc, effort), tuple(history))
```

## run_lab.py

```
"""Run the Lab 8 COCOMO parameter search and export reproducible results."""
from __future__ import annotations


import argparse
import csv
import json
import math
from pathlib import Path
import random
import statistics
import sys
import time


from PIL import Image, ImageDraw, ImageFont


from cocomo_ga import FitResult, absolute_error_rms, fit_genetic_algorithm, predict


ROOT = Path(__file__).resolve().parent
DATA = ROOT / "data" / "projects_18.csv"
RESULTS = ROOT / "results"
SPLIT_SEED = 3  # fixed split, leaves the unusually large source value visible in holdout
```

```
def load_data(path: Path = DATA) -> list[dict[str, float | int]]:
    with path.open(newline="", encoding="utf-8-sig") as stream:
        rows = list(csv.DictReader(stream))
    if len(rows) != 18:
        raise ValueError(f"Expected 18 source projects, got {len(rows)}")
    return [{"id": int(r["project_id"]), "kloc": float(r["kloc"]), "effort": float(r["effort_person_months"])} for r in
rows]



def split_data(rows: list[dict[str, float | int]], seed: int = SPLIT_SEED):
    indexes = list(range(len(rows)))
    random.Random(seed).shuffle(indexes)
    test_indexes = sorted(indexes[:5])
    train_indexes = sorted(indexes[5:])
    return [rows[i] for i in train_indexes], [rows[i] for i in test_indexes]



def metrics(actual: list[float], predicted: list[float]) -> dict[str, float]:
    if len(actual) != len(predicted) or not actual:
        raise ValueError("Metric vectors must have equal, non-zero lengths")
    errors = [abs(x - y) for x, y in zip(actual, predicted)]
    relative = [e / abs(x) for e, x in zip(errors, actual) if x != 0]
    return {
        "ED_RMS_person_months": absolute_error_rms(actual, predicted),
        "MD_sum_absolute_person_months": sum(errors),
        "MMRE_percent": 100.0 * statistics.mean(relative),
        "max_absolute_error_person_months": max(errors),
    }



def log_linear_baseline(train: list[dict[str, float | int]]):
    # Least-squares regression on ln(Ef) = ln(a) + b*ln(L); comparison only.
    xs = [math.log(float(r["kloc"])) for r in train]
    ys = [math.log(float(r["effort"])) for r in train]
    xm, ym = statistics.mean(xs), statistics.mean(ys)
    b = sum((x - xm) * (y - ym) for x, y in zip(xs, ys)) / sum((x - xm) ** 2 for x in xs)
    a = math.exp(ym - b * xm)
    return a, b



def _font(size: int, bold: bool = False):
    paths = [
        Path(r"C:\Windows\Fonts\arialbd.ttf" if bold else r"C:\Windows\Fonts\arial.ttf"),
        Path(r"C:\Windows\Fonts\calibrib.ttf" if bold else r"C:\Windows\Fonts\calibri.ttf"),
    ]
    for path in paths:
        if path.exists():
            return ImageFont.truetype(str(path), size)
    return ImageFont.load_default()



def _new_chart(title: str, subtitle: str, width: int = 1100, height: int = 650):
    image = Image.new("RGB", (width, height), "white")
    draw = ImageDraw.Draw(image)
    draw.text((width // 2, 24), title, fill="#17243a", font=_font(28, True), anchor="ma")
    draw.text((width // 2, 64), subtitle, fill="#596579", font=_font(16), anchor="ma")
    return image, draw



def chart_convergence(history: tuple[float, ...], output: Path):
    image, draw = _new_chart("Сходимость генетического алгоритма", "Лучшее значение ED на обучающей выборке")
    x0, y0, x1, y1 = 100, 120, 1030, 550
    data = [max(v, 1e-9) for v in history]
    lo, hi = min(data), max(data)
    loglo, loghi = math.log10(lo), math.log10(hi)
    for k in range(5):
        y = y0 + k * (y1 - y0) / 4
        draw.line((x0, y, x1, y), fill="#e3e8ef", width=1)
        value = 10 ** (loghi - k * (loghi - loglo) / 4)
        draw.text((x0 - 12, y), f"{value:.2g}", fill="#596579", font=_font(14), anchor="rm")
    pts = []
    for i, value in enumerate(data):
        x = x0 + i * (x1 - x0) / max(1, len(data) - 1)
```

```
        y = y1 - (math.log10(value) - loglo) / max(1e-12, loghi - loglo) * (y1 - y0)
        pts.append((x, y))
    draw.line(pts, fill="#2768b2", width=4)
    for label, x, y in [("0", x0, y1 + 18), (str(len(data) - 1), x1, y1 + 18)]:
        draw.text((x, y), label, fill="#596579", font=_font(14), anchor="ma")
    draw.text((565, 610), "Поколение", fill="#27364b", font=_font(17), anchor="ma")
    image.save(output)



def chart_predictions(train, test, model_values, baseline_values, output: Path):
    image, draw = _new_chart("Фактические и прогнозные трудозатраты", "Логарифмические оси; заполненные маркеры — GA,
контурные — регрессия")
    x0, y0, x1, y1 = 120, 130, 1015, 535
    actual = [float(r["effort"]) for r in train + test]
    all_values = actual + model_values + baseline_values
    lo = max(0.1, min(all_values) * 0.75)
    hi = max(all_values) * 1.15
    log_lo, log_hi = math.log10(lo), math.log10(hi)
    def sx(v): return x0 + (math.log10(max(lo, v))-log_lo)/(log_hi-log_lo)*(x1-x0)
    def sy(v): return y1 - (math.log10(max(lo, v))-log_lo)/(log_hi-log_lo)*(y1-y0)
    for k in range(6):
        value = 10 ** (log_lo + k*(log_hi-log_lo)/5)
        x, y = sx(value), sy(value)
        draw.line((x,y0,x,y1),fill="#e8edf3",width=1); draw.line((x0,y,x1,y),fill="#e8edf3",width=1)
        draw.text((x,y1+12),f"{value:.2g}",fill="#596579",font=_font(13),anchor="ma")
        draw.text((x0-10,y),f"{value:.2g}",fill="#596579",font=_font(13),anchor="rm")
    draw.line((sx(lo),sy(lo),sx(hi),sy(hi)),fill="#7e8999",width=2)
    all_rows=train+test
    for index,row in enumerate(all_rows):
        color="#2768b2" if index < len(train) else "#d77a24"
        x=sx(float(row["effort"]))
        for val,outline,offset in ((model_values[index],False,-4),(baseline_values[index],True,4)):
            y=sy(val)+offset
            draw.ellipse((x-6,y-6,x+6,y+6),fill="white" if outline else color,outline=color,width=2)
    draw.text((565,610),"Фактическое значение, чел.-мес.",fill="#27364b",font=_font(16),anchor="ma")
    draw.text((35,330),"Прогноз",fill="#27364b",font=_font(16),anchor="mm")
    draw.text((800,96),"Синее — обучение; оранжевое — тест",fill="#596579",font=_font(14))
    image.save(output)


def export_results(rows, train, test, result: FitResult, run_rows, baseline, elapsed: float):
    RESULTS.mkdir(exist_ok=True)
    a, b = result.a, result.b
    train_x = [float(r["kloc"]) for r in train]
    test_x = [float(r["kloc"]) for r in test]
    train_y = [float(r["effort"]) for r in train]
    test_y = [float(r["effort"]) for r in test]
    train_pred = predict(train_x, a, b)
    test_pred = predict(test_x, a, b)
    baseline_train = predict(train_x, *baseline)
    baseline_test = predict(test_x, *baseline)


    with (RESULTS / "ga_runs.csv").open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.DictWriter(stream, fieldnames=["seed", "a", "b", "train_ED"])
        writer.writeheader(); writer.writerows(run_rows)
    with (RESULTS / "predictions.csv").open("w", newline="", encoding="utf-8-sig") as stream:
        writer = csv.writer(stream)
        writer.writerow(["project_id",   "split",   "kloc",   "actual_effort",   "ga_prediction",   "ga_absolute_error",
"ga_MRE_percent", "baseline_prediction"])
        for split, group, predictions, base_predictions in [("train", train, train_pred, baseline_train), ("test", test,
test_pred, baseline_test)]:
            for r, p, bp in zip(group, predictions, base_predictions):
                actual = float(r["effort"])
                writer.writerow([r["id"], split, r["kloc"], actual, p, abs(actual-p), 100*abs(actual-p)/actual, bp])


    test_no_outlier = [(r, p) for r,p in zip(test,test_pred) if int(r["id"]) != 4]
    without_outlier = metrics([float(r["effort"]) for r,p in test_no_outlier],[p for r,p in test_no_outlier]) if
test_no_outlier else None
    summary = {
        "model": "Ef = a * L^b",
        "fitness": "ED = sqrt(mean((actual - predicted)^2))",
        "split_seed": SPLIT_SEED,
        "training_project_ids": [int(r["id"]) for r in train],
        "test_project_ids": [int(r["id"]) for r in test],
        "source_anomaly_note": "Project 4 is preserved exactly as printed in the assignment: effort 909.8 person-
months.",
        "ga": {"a": a, "b": b, "formula": f"Ef = {a:.6f} * L^{b:.6f}", "training_metrics": metrics(train_y,train_pred),
"test_metrics":   metrics(test_y,test_pred),   "test_metrics_excluding_project_4":   without_outlier,   "generations":
len(result.history)-1},
        "log_linear_baseline": {"a": baseline[0], "b": baseline[1], "training_metrics": metrics(train_y,baseline_train),
"test_metrics": metrics(test_y,baseline_test)},
        "repeat_runs": run_rows,
        "elapsed_seconds": elapsed,
    }
    (RESULTS / "summary.json").write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
```

```
    chart_convergence(result.history, RESULTS / "convergence.png")
    chart_predictions(train, test, train_pred+test_pred, baseline_train+baseline_test, RESULTS /
"actual_vs_predicted.png")
    return summary



def run_experiment(runs: int = 8, verbose: bool = False):
    rows = load_data()
    train, test = split_data(rows)
    x = [float(r["kloc"]) for r in train]
    y = [float(r["effort"]) for r in train]
    run_rows = []
    best = None
    start = time.perf_counter()
    for run in range(runs):
        seed = 42 + run
        result = fit_genetic_algorithm(x, y, population_size=120, generations=600, crossover_probability=0.9,
mutation_probability=0.15, seed=seed)
        record = {"seed": seed, "a": result.a, "b": result.b, "train_ED": result.error}
        run_rows.append(record)
        if best is None or result.error < best.error:
            best = result
        if verbose:
            print(f"Запуск {run+1:02d}/{runs}: a={result.a:.6f}, b={result.b:.6f}, ED train={result.error:.4f}")
    elapsed = time.perf_counter() - start
    baseline = log_linear_baseline(train)
    summary = export_results(rows, train, test, best, run_rows, baseline, elapsed)
    print("\nМодель COCOMO: Ef = a * L^b")
    print(f"Разбиение ({SPLIT_SEED=}): обучение {len(train)} проектов, тест {len(test)} проектов")
    print(f"GA: a={best.a:.6f}; b={best.b:.6f}; {summary['ga']['formula']}")
    print(f"ED обучения: {summary['ga']['training_metrics']['ED_RMS_person_months']:.3f} чел.-мес.")
    print(f"ED теста:    {summary['ga']['test_metrics']['ED_RMS_person_months']:.3f} чел.-мес.")
    print(f"Тест без проекта 4: {summary['ga']['test_metrics_excluding_project_4']['ED_RMS_person_months']:.3f} чел.-
мес.")
    print(f"Лог-линейная база: a={baseline[0]:.6f}; b={baseline[1]:.6f}")
    print(f"Результаты и графики: {RESULTS}")
    return summary



def menu():
    print("Лабораторная работа 8. Генетическая настройка COCOMO")
    print("Вариант 4: вещественное кодирование, ED, арифметические операторы, рулетка")
    while True:
        print("\n1 — выполнить серию экспериментов\n2 — показать параметры метода\n0 — выход")
        choice = input("Выберите пункт: ").strip()
        if choice == "1":
            run_experiment(verbose=True)
        elif choice == "2":
            print("Размер популяции: 120; поколения: 600; Pc=0.90; Pm=0.15; 8 независимых запусков.")
            print("Выбор: рулетка с преобразованием ошибки в положительные веса; сохранение элиты.")
        elif choice == "0":
            print("Работа завершена.")
            return
        else:
            print("Введите 1, 2 или 0.")



if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Вариант 4: оценка параметров COCOMO генетическим алгоритмом")
    parser.add_argument("--run", action="store_true", help="выполнить эксперименты без диалогового меню")
    parser.add_argument("--runs", type=int, default=8, help="число независимых запусков GA")
    parser.add_argument("--verbose", action="store_true", help="показывать итоги каждого запуска")
    args = parser.parse_args()
    if args.runs < 1:
        parser.error("--runs must be positive")
    run_experiment(args.runs, args.verbose) if args.run else menu()
```